

Full-Stack System Roadmap & Phased Implementation Plan

An end-to-end technical blueprint detailing the chronological execution sequence for the AI Cyber Guardian project: covering real-time WebSocket pipelines, machine learning training, enterprise deception honeypots, sample app integration, cloud containerization, and final defense evaluation.

PROJECT TITLE: AI Cyber Guardian (2026–2027)	VERSION: 2.4 Enterprise Edition	REPOSITORY: Prashant9998/AI-Gardian-2027
PRIMARY STACK: FastAPI · React 19 · Redis · Docker	SYSTEM STATUS: Phase 0 Baseline Verified	IMMEDIATE GOAL: Step 1: Real-Time WebSockets

EXECUTIVE ABSTRACT:
AI Cyber Guardian represents a next-generation web application firewall (WAF) and autonomous defense system. Rather than relying strictly on static regex rules or passive log analysis, it deploys a two-stage hybrid pipeline: Stage 1 evaluates sliding-window rate limiting in sub-millisecond Redis memory; Stage 2 performs dual machine learning threat fusion (Isolation Forest anomaly detection + Random Forest classification) to produce a composite threat score (0–100). Critical attackers are transparently routed into a virtual Linux honeypot sandbox where decoy canary tokens and an adaptive tarpit exhaust attacker resources while streaming real-time forensic intelligence into a React 19 SOC dashboard.

1. Current Baseline Status (Phase 0 — 100% Completed)

The core foundation, backend detection algorithms, enterprise deception subsystem, and SOC dashboard have been constructed and verified with automated test suites. Remote GitHub repository synchronization is complete.

Subsystem	File / Component Path	Technical Scope	Status
Rule Engine	core/rules.py	R1–R6 deterministic rules with double URL-decoding (SQLi, XSS, CmdInj, Path Traversal, Scanners, Brute Force)	✓ 100% Passed
Rate Limiting	core/rate_limiter.py	Sliding-window Redis rate limiter with automatic in-memory fail-open fallback	✓ 100% Passed
Threat Fusion	core/decision_engine.py	Hybrid score weighting formula: $\min(100, \text{rule_score} + \text{ml_score} * 30)$ mapped to 4 response tiers	✓ 100% Passed
Honeypot Subsystem	honeypot/*	Virtual Linux File System (fake shell), Canary tokens (AWS, DB, JWT), Lures, Adaptive Tarpit, STIX 2.1	✓ 100% Passed
React SOC Dashboard	cyberprep-app/src/*	Landing page, Login, and 10 Dashboard Views (Overview, Live Feed, Analytics, Geo Radar, Blocked IPs, Attackers)	✓ Live Preview
Docker Stack	docker-compose.yml	Multi-container orchestration: PostgreSQL 15, Redis 7, Backend FastAPI, Frontend React/Nginx	✓ Dockerized

2. Autonomous Defense Lifecycle

Traffic entering through the client SDK passes through successive defensive stages designed to reject malicious traffic without adding latency to legitimate client sessions:

Stage	Engine Component	Execution Model & Latency	Primary Outcome
Stage 1: Ingress	IP Filter & Redis Rate Limiter	Sub-millisecond (< 1ms) in-memory sliding window lookup	Drop CIDR blacklists & brute-force bursts before compute overhead
Stage 2: Fusion	Deterministic Rules + ML Anomaly Engine	Parallel execution with 120ms timeout safeguard	Composite Threat Score (0–100) combining signatures & zero-day ML
Stage 3: Action	Autonomous Decision Engine	Instant threshold evaluation	LOW (Log), MEDIUM (Alert), HIGH (Block 403), CRITICAL (Honeypot)
Stage 4: Deception	Enterprise Honeypot & Tarpit	Stateful VFS Linux sandbox + injected latency	Exhaust attacker automation while capturing STIX 2.1 forensic data

3. Full-Stack Phased Master Implementation Plan

The following 8 steps are organized in exact chronological sequence. Each step establishes the mandatory foundation for subsequent phases, ensuring continuous stability and testability.

STEP 1: REAL-TIME WEBSOCKET STREAMING PIPELINE

IMMEDIATE PRIORITY —
SUB-100MS LATENCY

Objective & Rationale: Connects the backend decision stream directly to the React dashboard. Instead of 3.5-second polling, live attacks appear on screen instantaneously.

Key Deliverables:

- **FastAPI WebSocket Server:** Implement `/api/v1/ws/threats` in `api/routers/ws.py` with an asynchronous `ConnectionManager` supporting multiple concurrent dashboard connections.
- **Event Interceptor:** Wire `evaluate_decision()` and `honeypot/engine.py` to broadcast security events as JSON payloads over active WebSockets immediately upon evaluation.
- **React useThreatStream Hook:** Create `cyberprep-app/src/hooks/useThreatStream.js` with auto-reconnection, exponential backoff, and direct state insertion into the Live Threat Feed.
- **Visual & Audio Alerts:** Implement an alert flash animation on new event arrival with an optional toggleable sound chime for CRITICAL severity attacks.

Acceptance Criteria: Sending an attack payload via curl displays the event on the React SOC Dashboard in < 100ms with zero manual page refreshes.

STEP 2: REAL ML DATASET TRAINING & MODEL SERIALIZATION

ACADEMIC & SCIENTIFIC
VALIDITY

Objective & Rationale: Replaces in-memory mock models with scikit-learn models trained on benchmark datasets (CSIC 2010 HTTP Dataset and CIC-IDS2017) to provide statistical rigor.

Key Deliverables:

- **Feature Extraction Pipeline:** Build `guardian-control-plane/ml/features.py` extracting length, entropy, special symbol ratios (', ", <, >, ,, --, /, \), parameter count, and TF-IDF char n-grams.
- **Offline Training Script:** Train an **Isolation Forest** (unsupervised anomaly detection) and a **Random Forest Classifier** (supervised multi-class categorizer) on CSIC 2010 data.
- **Model Serialization:** Export trained weights to `isolation_forest.joblib`, `random_forest.joblib`, and `vectorizer.joblib`.
- **FastAPI Lifespan Loading:** Load serialized joblib models into memory during FastAPI startup in < 500ms.

Acceptance Criteria: Model achieves $\geq 92\%$ F1-score and $\leq 4.5\%$ False Positive Rate on clean HTTP requests with verified Confusion Matrix benchmarks.

STEP 3: PROTECTED TARGET APPLICATION LAB ("PROTECTED APP LAB")

REAL APPLICATION
INTEGRATION

Objective & Rationale: Demonstrates the Guardian protecting an actual functional web application (e.g. E-Commerce or Banking Portal) in a real-world scenario.

Key Deliverables:

- **Sample App Development:** Create `sample-target-app/` featuring realistic endpoints: `POST /api/login`, `GET /api/products/search`, `POST /api/feedback`, and `GET /download`.
- **SDK Middleware Attachment:** Attach the Guardian SDK middleware in 3 lines of code: `app.use(guardianMiddleware(...))`.
- **End-to-End Test Matrix:** Verify that benign searches return 200 OK, SQLi searches are blocked with 403 Forbidden, and access to decoy `/env` routes to the Honeypot.

Acceptance Criteria: Attacks against the target application are intercepted transparently without impacting legitimate end-user application workflows.

STEP 4: 1-CLICK INTERACTIVE ATTACK DEMO HARNESS

KEY EVALUATION & PRESENTATION ASSET

Objective & Rationale: Enables an automated multi-stage Red Team vs. Blue Team attack scenario that evaluators can launch with a single button during project defense.

Key Deliverables:

- **Multi-Stage Attack Runner:** Build `simulate_demo_scenario.py` executing 4 distinct threat phases: Recon Scan (0–5s) → Brute Force (5–12s) → SQLi Exploit (12–20s) → Honeypot VFS Shell Exploration & Canary Tripping (20–30s).
- **SOC Dashboard Launch Trigger:** Add an interactive '■ Run Live Attack Demo' button on the Dashboard Topbar that triggers the scenario and highlights corresponding dashboard panels sequentially.

Acceptance Criteria: Evaluators can press one button and watch the full defense lifecycle (detect, block, trap, report) unfold autonomously on the live dashboard.

STEP 5: OFFICIAL PYTHON SDK PACKAGE (CYBER-GUARDIAN)

ECOSYSTEM REUSABILITY

Objective & Rationale: Completes the promise on the Landing Page ('pip install cyber-guardian') by providing native middleware for FastAPI, Flask, and Django.

Key Deliverables:

- **Python Package Architecture:** Set up guardian-sdk-python/ with pyproject.toml supporting pip install -e ..
- **In-Memory Decision Cache:** Implement LRU caching with TTL for clean IP decisions to prevent unnecessary network latency on repetitive legitimate requests.
- **Framework Adapters:** Implement BaseHTTPMiddleware for FastAPI/Starlette, before_request hooks for Flask, and MiddlewareMixin for Django.
- **Asynchronous Telemetry:** Non-blocking background worker dispatching security telemetry without delaying client response times (< 2ms overhead).

Acceptance Criteria: Any Python developer can install cyber-guardian via pip and protect their application with 3 lines of middleware code.

STEP 6: DATABASE PERSISTENCE & REAL PDF THREAT REPORT GENERATOR

COMPLIANCE & AUDITABILITY

Objective & Rationale: Security audits require permanent event records and downloadable executive summaries with incident timelines and STIX indicators.

Key Deliverables:

- **Database Migrations:** Configure Alembic to manage ThreatEvent, AttackerProfile, BlockedIP, and HoneypotSession schemas in PostgreSQL.
- **Timeseries Aggregation:** Create high-performance indexed queries for 24-hour attack trends, top attacker nations, and vector breakdown percentages.
- **Server-Side PDF Exporter:** Implement /api/v1/reporting/export-pdf compiling incident charts, top attacker profiles, and STIX threat indicators into an executive PDF.

Acceptance Criteria: Clicking 'Generate PDF Report' in the React dashboard downloads a polished, multi-page PDF summary compiled from real database records.

STEP 7: PRODUCTION CLOUD DEPLOYMENT & CI/CD PIPELINE

PUBLIC AVAILABILITY

Objective & Rationale: Deploys the containerized stack to a public cloud VPS (DigitalOcean / AWS / GCP) with automated SSL encryption for remote viva demonstrations.

Key Deliverables:

- **Cloud Server Provisioning:** Deploy to an Ubuntu 22.04 LTS instance with Docker and Docker Compose.
- **Caddy Edge Reverse Proxy:** Configure Caddy for automated Let's Encrypt TLS certificate issuance and WebSocket connection termination.
- **GitHub Actions CI/CD:** Set up .github/workflows/ci.yml running Pytest tests and Vite build checks on every push to maintain zero regression.

Acceptance Criteria: The complete platform is live and publicly accessible over HTTPS (e.g. https://guardian.yourdomain.com) with automated health recovery.

STEP 8: ACADEMIC DEFENSE, VIVA PACKAGE & RESEARCH PAPER

PROJECT EVALUATION & GRADING

Objective & Rationale: Prepares all formal academic documentation, research publications, and viva answers required for top university project evaluations.

Key Deliverables:

- **Final Project Report:** Comprehensive thesis covering Abstract, Literature Review, Threat Modeling, Architecture, Benchmarks, and Future Work.
- **IEEE Conference Paper Draft:** Formatted 6-page paper: '*Autonomous Real-Time Web Application Defense using Machine Learning Threat Fusion and Stateful Deception Honeypots*'.
- **Defense Presentation Deck:** 15-slide PowerPoint deck covering problem statement, threat matrix, architecture flowcharts, and live demo results.
- **Viva Q&A; Cheat Sheet:** In-depth answers to tough examiner questions on adversarial evasion, fail-open design, and rate-limiting concurrency.

Acceptance Criteria: Complete project documentation and viva defense kit ready for university submission with zero remaining gaps.

4. Threat Decision Matrix & Response Protocols

Incoming traffic is classified into four operational severity tiers based on the fused threat score:

Score Band	Severity	Trigger Conditions & Signatures	Autonomous Mitigation Action	Telemetry Logged
0 – 29	LOW	Normal browsing, valid form inputs, standard User-Agents, score < 30	Pass through untouched. Minimal audit logging to preserve performance.	Timestamp, path, latency, IP hash
30 – 59	MEDIUM	Automated scanners (Nikto, DirBuster), parameter fuzzing, minor anomalies	Pass through + Flag Session. Email alert triggered if operator enabled.	Full headers, query params, tool fingerprint
60 – 84	HIGH	Brute force (> 15 failed logins in 120s), Path Traversal (../etc/passwd), XSS	Immediate 403 Forbidden Block. IP placed in Redis temporary ban list.	IP, violation rules, payload snippet, ban TTL
85 – 100	CRITICAL	SQLi (UNION/SELECT), Command Injection, decoy lures (/env, /wp-login.php)	SILENT HONEYPOT ROUTING. Attacker trapped in Virtual Linux VFS shell; Canary tokens deployed; Tarpit latency injected.	Shell command history, keystroke timing, canary exfil, STIX 2.1 JSON

5. Architectural Reliability & Risk Mitigation

Risk / Failure Scenario	Severity	Engineered Architectural Safeguard
Redis Cache Outage	HIGH	Fail-Open Design: Rate limiter logs a CRITICAL error and falls back to an in-memory dictionary. Legitimate site traffic stays online rather than taking down the business app.
ML Inference Latency Spike	MEDIUM	Asynchronous 120ms Guard: If ML evaluation exceeds 120ms, the deterministic rule score takes immediate precedence, ensuring sub-second response times.
Attacker Honeypot Escape	CRITICAL	Pure Virtual Sandbox (VFS): The shell runs entirely in Python memory data structures. No real OS subshells (exec, os.system) are ever spawned, making breakout impossible.
False Positive Disruption	MEDIUM	Rule Override & Exclusion Paths: Operators can configure route exclusion lists (e.g. blog editors, Markdown fields) and adjust threshold weights via thresholds.json without restarting services.